EcoCode: Green-Software Analysis Platform

Prototype-Stage Project Report for UCS503P
Submitted to: Dr. Raghav B. Venkataramaiyer
Thapar Institute of Engineering and Technology

Gurnoor Singh Sodhi* Rakesh† Manav‡

September 2026

*Roll No: 1024030387
†Roll No: 1024030393
‡Roll No: 1024030398

Contents

1 Project Overview **3**

2 Higher-order Goal *green-software framing* **3**

3 Problem Statement **3**

4 Time-to-Value and Iterative Delivery **3**

5 Current Architecture **3**

6 Analysis Phase **4**
6.1 Use Cases 4
6.2 Activity / Swimlane Diagram 5
6.3 Class Diagram 6
6.4 Data Flow Diagrams 6
6.4.1 DFD Level 0 6
6.4.2 DFD Level 1 6
6.4.3 DFD Level 2 7

7 Software Requirement Specification *IEEE-style* **7**
7.1 Purpose 7
7.2 Scope 7
7.3 Functional Requirements 7
7.4 Non-Functional Requirements 8

8 Rule Engine **8**

9 Prototype Validation **8**
9.1 Rule Counts on the Reference Run 9

10 Evaluation Criterion **9**

11 User Stories and Story Cards **9**

12 SDLC Model **10**

13 Current Limitations **10**

14 Next Iteration **10**

15 Team Contribution Summary **10**

16 Conclusion **11**

1 Project Overview

EcoCode is a deterministic static-analysis prototype for Java projects. Its immediate goal is to identify source-code patterns that may contribute to unnecessary computation, repeated allocation, expensive collection operations, deep call chains, resource-management risks, or other forms of avoidable software resource usage.

The current prototype deliberately focuses on **whole-project static analysis**. It does not claim that every static finding is a confirmed performance defect. Instead, findings are treated as review candidates whose precision is being improved through testing and false-positive analysis.

2 Higher-order Goal

green-software framing

The higher-order goal is to support more resource-efficient software engineering. EcoCode translates this broad sustainability objective into an engineering problem that can be measured: locate potentially wasteful source-code patterns, attach source-level evidence, and later validate whether recommended changes measurably improve runtime behaviour.

3 Problem Statement

Modern Java applications can contain inefficient patterns that are difficult to identify through manual review alone, particularly when projects span many files and classes. Examples include repeated linear lookups inside loops, repeated object allocation on frequently executed paths, resource-management risks, unnecessary duplicate computation, and deep delegation chains.

Existing profilers are useful after software is running, but the team wanted an early-stage engineering tool that can inspect a complete repository before runtime profiling and produce deterministic, explainable findings tied to source files and methods.

4 Time-to-Value and Iterative Delivery

EcoCode follows an iterative and incremental development model so that each stage produces a usable engineering increment.

- **Iteration 1 – Foundation:** Gradle setup, project indexing, Java file metadata, JavaParser integration, whole-project parsing, ProjectModel.
- **Iteration 2 – Static Relationships:** method-call extraction, lightweight symbol-resolution layer, call-graph construction, reachability/depth support.
- **Iteration 3 – Rule Engine:** EcoRule abstraction, RuleEngine, EC101–EC108, JUnit tests, open-source repository validation, false-positive refinement.
- **Next iteration:** build automation and runtime intelligence, including containerized execution and profiling evidence.

This structure supports the course emphasis on fast build-test-feedback cycles rather than waiting for a single final implementation.

5 Current Architecture

Figure 1 summarizes the current Iteration-3 analysis pipeline.



Figure 1: EcoCode Iteration-3 architecture

The active prototype pipeline is:

Project Path → ProjectIndexer → JavaProjectParser → ProjectModel → SymbolSolver → CallGraph → RuleEngine → Static Findings.

The current `SymbolSolver` is a **lightweight deterministic resolver**; it should not be described as full JavaParser semantic symbol solving.

6 Analysis Phase

6.1 Use Cases

The main actor is a developer or evaluator who selects a Java project and requests analysis. EcoCode indexes the repository, parses source files, constructs an internal model, extracts static relationships, executes deterministic rules, and reports source-linked findings.

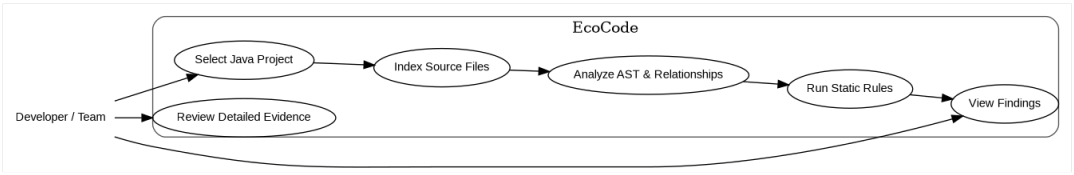
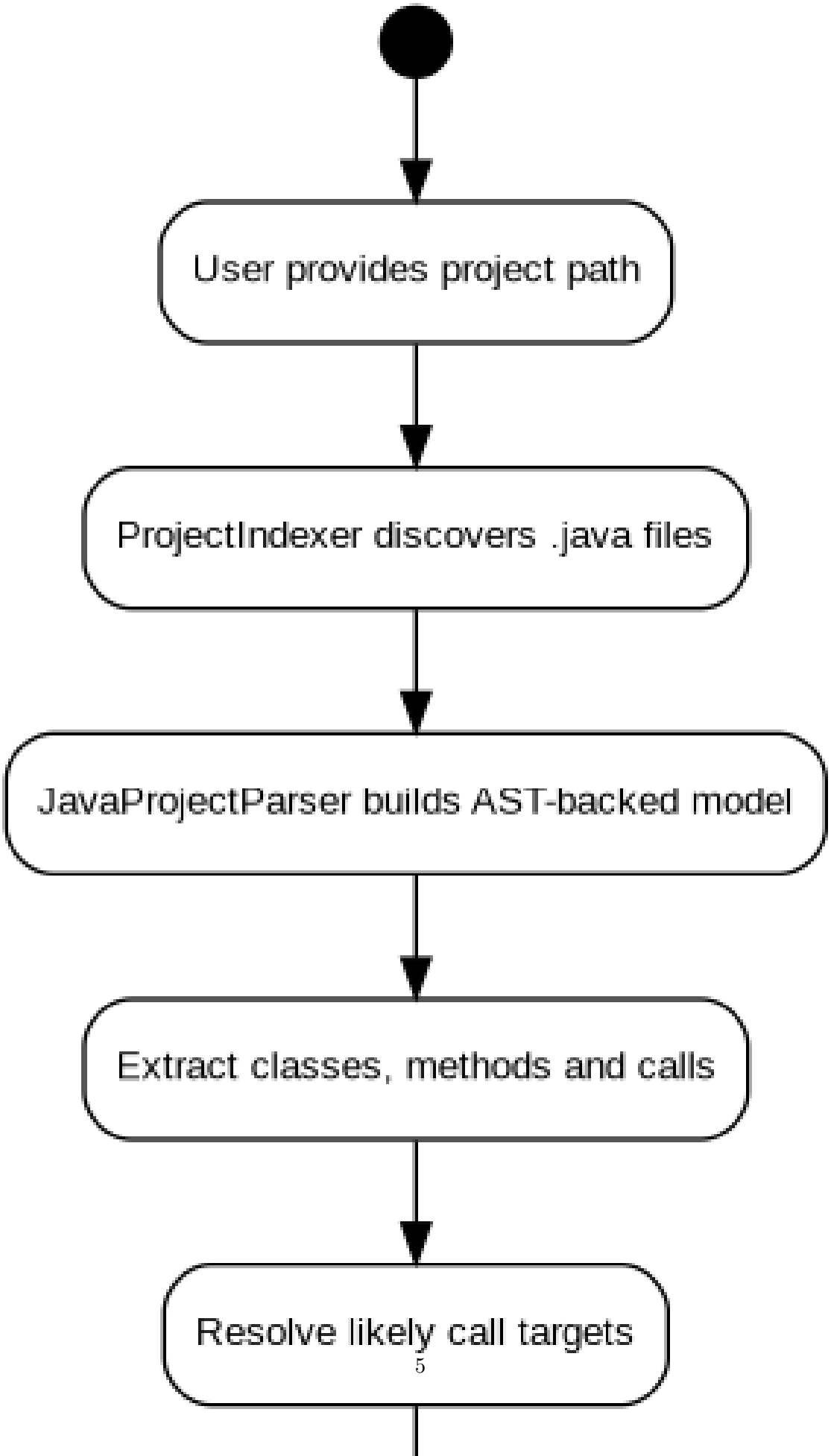


Figure 2: Use-case diagram

6.2 Activity / Swimlane Diagram



6.3 Class Diagram

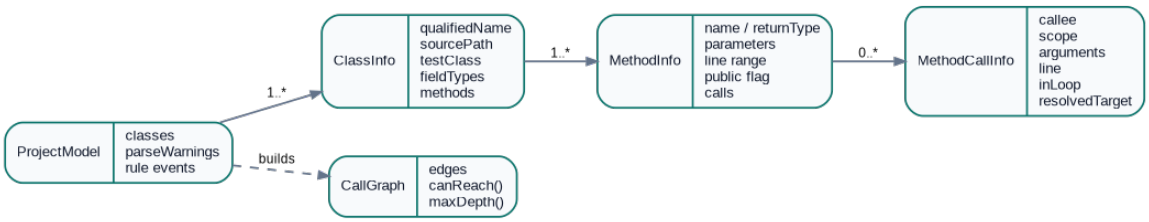


Figure 4: Core class model for the static-analysis prototype

The main classes include `ProjectIndexer`, `JavaFileInfo`, `JavaProjectParser`, `ProjectModel`, `ClassInfo`, `MethodInfo`, `MethodCallInfo`, `SymbolSolver`, `CallGraph`, `EcoRule`, `RuleEngine`, the eight EC rules, and `StaticIssue`.

6.4 Data Flow Diagrams

6.4.1 DFD Level 0

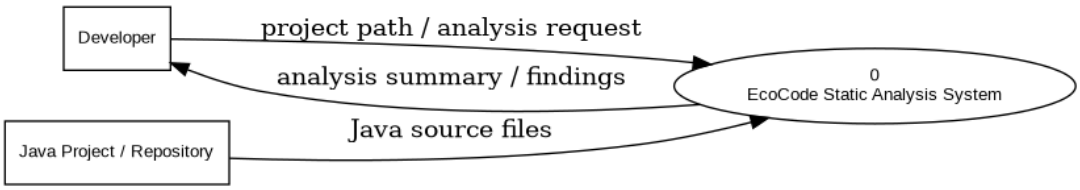


Figure 5: DFD Level 0

6.4.2 DFD Level 1

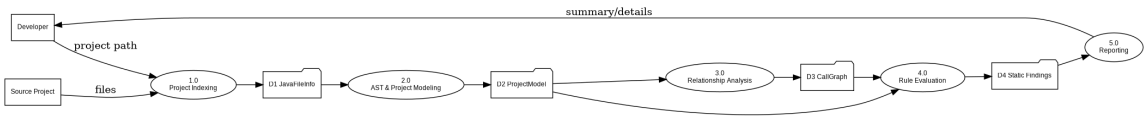


Figure 6: DFD Level 1

6.4.3 DFD Level 2

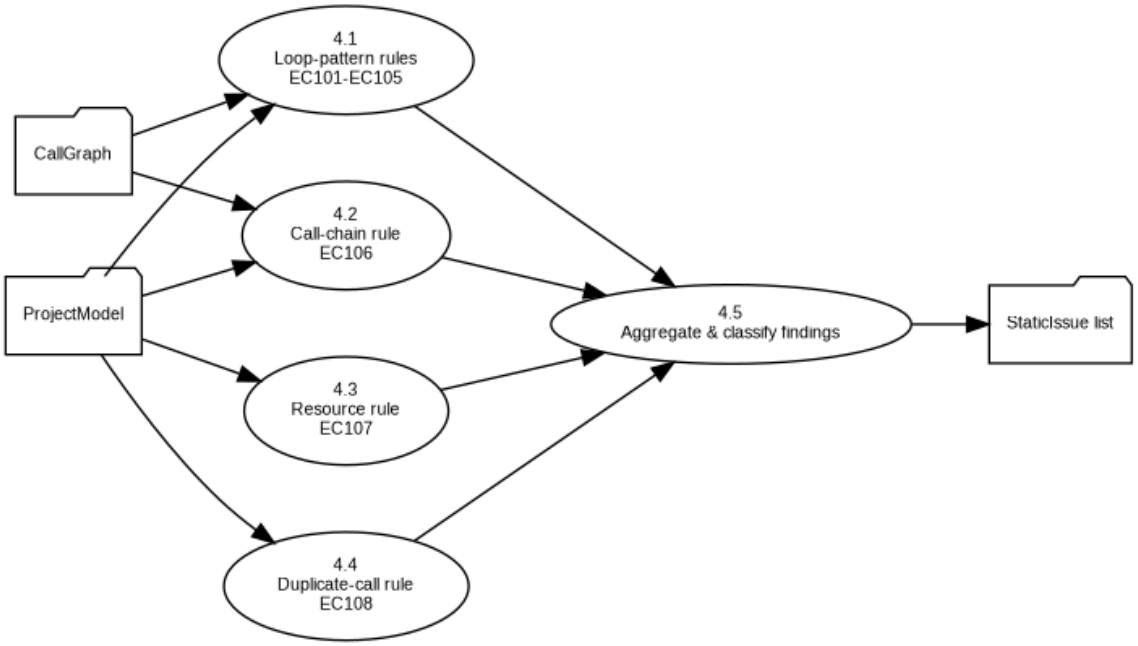


Figure 7: DFD Level 2

7 Software Requirement Specification *IEEE-style*

7.1 Purpose

The purpose of EcoCode is to analyse complete Java projects and produce deterministic, explainable static findings that may indicate opportunities for resource-efficiency improvement.

7.2 Scope

The prototype currently supports local Java-project analysis. It indexes source files, parses Java source into ASTs, builds a project model, extracts method calls, constructs a prototype call graph, and applies EC101–EC108. Runtime profiling, evidence confirmation, version comparison, and production deployment are future-stage features.

7.3 Functional Requirements

ID	Requirement
FR1	The system shall accept a local Java project path as input.
FR2	The system shall recursively identify Java source files.
FR3	The system shall record file metadata including line count and test/non-test classification.
FR4	The system shall parse Java source files and build a whole-project model.
FR5	The system shall extract classes, methods, method calls, and source locations.
FR6	The system shall apply a lightweight deterministic symbol-resolution layer.
FR7	The system shall construct a prototype call graph from extracted method relationships.

FR8	The system shall execute EC101–EC108 through a common rule-engine abstraction.
FR9	The system shall report rule ID, file, line, class, method, severity/confidence, explanation, and recommendation where available.
FR10	The system shall distinguish test-code findings from non-test source findings.

7.4 Non-Functional Requirements

- **Explainability:** Findings should be deterministic and source-linked.
- **Maintainability:** Rules should implement a shared abstraction so they can be refined independently.
- **Scalability:** The architecture should extend from small repositories to larger multi-file Java projects.
- **Reproducibility:** The same source tree and rule version should produce repeatable static findings.
- **Portability:** The prototype targets Java 21 and Gradle-based execution.

8 Rule Engine

The current rule set is shown below. These rules are implemented but are **still under testing**; their thresholds and false-positive behaviour are not considered final.

Rule	Current interpretation
EC101	Repeated database/repository call in a loop
EC102	Accumulating string concatenation in a loop
EC103	Linear collection lookup in a loop
EC104	Potential unbounded collection fetch
EC105	Potential repeated object allocation in a loop
EC106	Deep call-chain review signal
EC107	Potential unclosed system resource
EC108	Potential duplicate computation

9 Prototype Validation

The current reference repository is `minimal-json`. The latest validated run produced the following structural metrics:

Metric	Result
Java files	54
Non-test source files	37
Test files	17
Lines of code	10,870
Classes	63
Methods	912
Method calls	2,931
Call-graph nodes	468
Parse warnings	0
Static findings	106
Non-test source findings	42
Test-code findings	64

9.1 Rule Counts on the Reference Run

Rule	Total	Non-test source
EC101	0	0
EC102	0	0
EC103	4	4
EC104	0	0
EC105	20	13
EC106	1	1
EC107	14	5
EC108	67	19

The team initially observed 285 findings before auditing the heuristics. Manual review exposed broad interpretations in some rules, and the rules were made more conservative. The latest reference run reports 106 findings. This reduction is treated as evidence of **false-positive refinement**, not as loss of functionality.

10 Evaluation Criterion

At the prototype stage, success is evaluated using measurable engineering evidence:

- complete parsing of the selected repository with zero parser warnings on the reference run;
- reproducible structural metrics (files, LOC, classes, methods, calls, graph nodes);
- automated JUnit tests for rule behaviour;
- manual inspection of representative findings;
- reduction of demonstrable false positives while preserving intended detections.

Later evaluation will include runtime measurements so potential findings can be confirmed or rejected with evidence.

11 User Stories and Story Cards

- **US1:** As a developer, I want to scan an entire Java repository so that I can review potential resource-efficiency issues without inspecting every file manually.

- **US2:** As a developer, I want each finding to include a rule ID, source file, method, and line so that I can verify the evidence quickly.
- **US3:** As a maintainer, I want rules to use a shared interface so that rules can be added, tested, or refined independently.
- **US4:** As an evaluator, I want project-level metrics and parse warnings so that I can judge whether the analyser actually processed the repository.
- **US5:** As a developer, I want test-code findings separated from non-test source findings so that analysis results are easier to interpret.

12 SDLC Model

EcoCode uses an **Agile Iterative and Incremental** model. Each iteration produces a working increment, is tested on real code, and feeds observed problems back into the next increment. This model is suitable because static-analysis heuristics improve through repeated build-test-review cycles rather than a single one-time implementation.

13 Current Limitations

- Rule testing and false-positive refinement are ongoing; rules are not finalised.
- Static findings are not equivalent to confirmed performance defects.
- The current symbol resolver is heuristic rather than full semantic JavaParser Symbol Solver integration.
- EC105, EC106, EC107, and EC108 are intentionally conservative review signals in the prototype.
- Runtime profiling and evidence confirmation are not part of the current Iteration-3 submission.

14 Next Iteration

The next planned stream moves from static intelligence toward runtime evidence: build automation, containerized execution, benchmark management, JVM profiling, and an evidence layer that can classify static findings as potential, confirmed, validated, or rejected.

15 Team Contribution Summary

- **Gurnoor Singh Sodhi:** static-analysis foundation, project indexing, JavaParser/project-model work, integration understanding, rule validation and false-positive audit, documentation and presentation preparation.
- **Rakesh:** runtime-intelligence stream ownership including build execution, Docker/runtime profiling, benchmark and system-metric work according to the team roadmap.
- **Manav:** integration/validation stream, version comparison, repository integration, UI/backend integration and testing according to the team roadmap.

16 Conclusion

EcoCode has reached a working Iteration-3 static-analysis prototype. It can process a real multi-file Java repository, build structural and call-relationship information, execute a deterministic eight-rule engine, and report source-linked findings. The team is deliberately presenting the rule set as **implemented but still under validation**. The next stage will add runtime evidence so the system can move from potential static patterns toward measurable green-software recommendations.